CDWIA v2 — Architecture Learning Guide

A component-by-component walkthrough: what it does, why it exists, what it cost to choose, and how to talk about it in an interview

---

How to use this document

Each section below covers one architectural element. Every section follows the same four-part structure, plus an interview section:

- **What it is** — the plain description
- **Why it exists** — the gap it closes or the failure mode it prevents
- **Design choices & alternatives** — what was chosen, what was rejected, and the actual tradeoff
- **Impact on the solution** — what breaks or degrades if this piece is removed
- **Interview questions & answers** — the questions this component tends to generate in a system design interview, answered the way you'd want to answer them

This mirrors how the components map to the codebase, if you're using this alongside the scaffold:

Doc section	Code location
Gateway	`src/cdwia/gateway/main.py`
Classifier / Planner / Policy / Cost Router	`src/cdwia/control_plane/`
SQL Agent (validator, executor)	`src/cdwia/data_plane/sql_agent/`
Knowledge Agent	`src/cdwia/data_plane/knowledge_agent/`
Hybrid Orchestrator	`src/cdwia/data_plane/hybrid/orchestrator.py`
Synthesizer & Guardrails	`src/cdwia/synthesizer/`
Ingestion	`src/cdwia/ingestion/pipeline.py`
Cache / Audit	`src/cdwia/common/`

---

1. The control plane / data plane split

What it is

The system is structured around two zones. The **control plane** makes decisions: authentication, authorization, classification, planning, and cost checks. The **data plane** executes: it runs SQL, retrieves documents, and returns raw results. The control plane sits *before* the data plane in the request path — not beside it.

Why it exists

Without this split, the natural thing to build is a single pipeline where "figure out what to do" and "do it" are interleaved — classify, then maybe generate SQL, then check if the user is allowed to run it, then run it. That ordering means you can spend an LLM call generating SQL for a request that was going to be denied anyway. Putting authorization and cost checks *before* any generation or retrieval work means you never pay for work you're going to throw away.

It also solves an operational problem: policy changes and routing logic (control plane) change slowly and must be provably correct. Query execution (data plane) is high-throughput and needs to autoscale independently. If they're one monolith, a policy update requires redeploying — and load-testing — the whole system.

Design choices & alternatives

- **Chosen:** strict separation, control plane fully gates entry to the data plane.
- **Rejected:** a single orchestration layer where checks are interleaved with execution (simpler to write initially, but couples scaling and deployment of two things with very different operational profiles).
- **Rejected:** control and data plane as peers with a shared decision point (this reintroduces the risk of a request reaching partial execution before being denied).

Impact on the solution

- Cost: denied or invalid requests are rejected before any LLM/compute spend.
- Scaling: the data plane (high throughput, execution-heavy) can scale horizontally without the control plane (low throughput, correctness-critical) needing to scale in lockstep.
- Change safety: a new OPA rule ships without touching the SQL execution path at all.

Interview questions & answers

Q: Why not just check authorization inside the SQL agent right before execution? A: You could, but then you've already spent the cost of classifying the query and generating SQL for a request that was never going to be allowed. Front-loading the check means you fail fast and cheap. It also means authorization logic lives in exactly one place instead of being re-implemented (and potentially inconsistently) inside every agent that executes something.

Q: Doesn't this add latency, since every request has to go through the control plane first? A: The control plane's checks (classification, policy lookup) are designed to be sub-50ms in the common case — a lightweight classifier and a cached/local policy decision, not an LLM call. The latency cost of "decide first" is much smaller than the cost of "generate then discard."

Q: How do you avoid the control plane becoming a bottleneck if it's not scaled the same way as the data plane? A: Because control plane work is cheap and deterministic for the majority of traffic (classifier hit, not LLM fallback), its throughput ceiling is much higher per instance than the data plane's. It still scales horizontally — it's just that its correctness requirements (careful review, testing, versioned policy) matter more than its raw scaling requirements, whereas the data plane's scaling requirements dominate.

---

2. API Gateway

What it is

The single entry point for all requests. Handles authentication (AuthN), rate limiting, and a WAF-style backstop against obviously malicious payloads.

Why it exists

Every request needs identity resolved exactly once, in exactly one place, before anything downstream trusts it. Without a gateway, each service would need to independently verify tokens — more code paths, more chances for one of them to get it wrong, and no single place to change token verification logic when the identity provider changes.

Design choices & alternatives

- **Chosen:** gateway does AuthN only (who are you); authorization (what can you do) is explicitly *not* the gateway's job — that's OPA in the control plane.
- **Rejected:** gateway does both AuthN and AuthZ. This was rejected because authorization needs context the gateway doesn't have cheaply (business unit, resource being requested, ABAC attributes) — conflating the two makes the gateway a second place authorization logic can drift from the control plane's.
- **Chosen:** rate limiting per-user at the gateway (cheap, fast, protects everything downstream) rather than per-service.

Impact on the solution

- If the gateway's AuthN is wrong (e.g. a bug lets an invalid token through), every downstream check that trusts `Principal` is compromised — this is why AuthN correctness here is disproportionately important.
- Removing the gateway means duplicating token verification and rate limiting into every service, or trusting the network boundary alone (fragile).

Interview questions & answers

Q: Why separate authentication from authorization instead of doing both at the edge? A: Authentication answers "who is this" and can be resolved from a token alone — this is cheap and doesn't need business logic. Authorization answers "can this identity do this specific thing," which depends on the resource, the action, and policy that changes independently of identity. Mixing them means every authorization policy change touches the same code path as login, which is a change-management hazard for something security-critical.

Q: What happens if the WAF-style filter in the gateway is bypassed? A: It's explicitly a backstop, not the only line of defense — the AST SQL validator downstream is what actually prevents SQL injection from mattering, because it operates on parsed structure, not string matching. The gateway's filter is meant to catch the cheap, obvious cases early; it's not the security boundary.

Q: Why rate-limit per-user rather than globally? A: A global limit lets one noisy user degrade the experience for everyone else. Per-user limiting isolates blast radius — the tradeoff is it requires tracking state per identity (here, a sliding window keyed on user ID), which needs to be centralized (e.g. Redis) once you have more than one gateway replica.

---

3. Control Plane: Deterministic Classifier

What it is

A lightweight, fast classifier that looks at the incoming question and decides whether it needs SQL, document retrieval, or both (hybrid) — before any LLM is invoked.

Why it exists

Most questions are structurally obvious. "Which team spent the most on EC2 last month" doesn't need an LLM to recognize as a SQL question. Running every single query through an LLM-based planner to make this decision is the single most avoidable cost in the whole system — it adds both latency (a network round trip to a model) and dollar cost (a token bill) to a decision that a regex or a small trained model can make in milliseconds.

Design choices & alternatives

- **Chosen:** deterministic classifier first, LLM planner as fallback only when confidence is low.
- **Rejected:** LLM-only planning for every request. Simpler to build, dramatically more expensive and slower at scale, and non-deterministic — the same question can get classified differently on different days, which makes debugging and testing much harder.
- **Note on the reference implementation:** the scaffold uses a regex heuristic as a stand-in. Production would replace this with a small trained model (e.g. a fine-tuned

sentence classifier) behind the exact same interface — the contract (`ClassificationResult` with a path and a confidence score) doesn't change.

Impact on the solution

- This is called out in the design doc as "the single highest-leverage cost and latency optimization in the whole system" — removing it means every request pays for an LLM call just to decide what to do next, before any actual work happens.
- Confidence scoring is what makes the fallback safe: low-confidence classifications defer to the LLM planner rather than guessing wrong silently.

Interview questions & answers

Q: Why not just always use the LLM for classification — isn't it more accurate? A: For the common case, no — a well-tuned lightweight classifier is often *more* consistent than an LLM for a narrow, well-defined classification task, because it's deterministic and testable with unit tests in a way that LLM outputs aren't. The LLM is reserved for the genuinely ambiguous cases where its flexibility earns its cost.

Q: How do you decide the confidence threshold for falling back to the LLM? A: It's a tunable value (`classifier_confidence_threshold`), set by looking at the classifier's precision/recall on a labeled dataset of real queries and picking the point where false "high confidence" classifications become rare enough to accept, balanced against how often you're willing to pay for the LLM fallback.

Q: What happens if the classifier is wrong and confidently so? A: The downstream agents are the safety net — the SQL agent's AST validator still runs even if the classifier's routing to "SQL" was wrong; a hybrid orchestrator that gets a document-only question can still run a (harmlessly empty) SQL path. The system is designed so a wrong route degrades the answer rather than causing incorrect execution.

---

4. Control Plane: LLM Fallback Planner

What it is

An LLM call invoked only when the deterministic classifier's confidence is below threshold, to make the final call on SQL vs. document vs. hybrid.

Why it exists

Some questions are genuinely ambiguous by structure alone — "our storage cost increased by 45%, how can we optimize it?" needs both a "what happened" (data) answer and a "what to do" (documentation) answer, and no simple lexical rule reliably detects that pattern across all phrasings. The LLM's language understanding is worth its cost specifically in this minority case.

Design choices & alternatives

- **Chosen:** LLM only as fallback, with a feature flag (`enable_llm_fallback`) to disable it entirely if needed (e.g. cost caps, provider outage).
- **Rejected:** LLM as the primary router (see classifier section — same tradeoff, cost/latency/determinism).
- Note the asymmetry in the code: the fallback planner's result always reports `used_llm_fallback=True` and a boosted confidence — this is deliberate, so downstream logging/auditing can distinguish "the deterministic classifier was confident" from "an LLM had to break the tie," which matters for debugging misclassifications later.

Impact on the solution

- Removing this means ambiguous queries get whatever the classifier's low-confidence default decides — which, in the reference implementation, defaults to hybrid. That's a safe default (hybrid tends to give supplementary coverage) but not always the *right* one.
- The feature flag matters operationally: if the LLM provider is degraded, you can disable fallback and accept the classifier's best guess rather than have every ambiguous query time out or fail.

Interview questions & answers

Q: If the LLM planner disagrees with the deterministic classifier, which one wins? A: The LLM planner's decision overrides the classifier's, but only because it was invoked in the first place due to low classifier confidence — the classifier is not overridden when it was already confident. This ordering matters: the fallback is a tie-breaker, not a second opinion on every request.

Q: What's the risk of an LLM-based planner in a production routing path? A: Non-determinism and latency variance. Two mitigations here: it's only on the low-confidence minority of traffic (bounding blast radius), and there's a hard fallback (`should_invoke_planner` returning false, or the flag being off) that keeps the system functional without it.

---

5. Control Plane: Policy Engine (OPA)

What it is

A thin client wrapping Open Policy Agent (OPA). Every request's action is checked against a declarative policy (written in Rego) before planning or execution proceeds.

Why it exists

Authorization logic that's hardcoded into application code is hard to review, hard to test in isolation, and requires a full redeploy of the service to change. Access control is also exactly

the kind of thing that needs an audit trail and sign-off separate from ordinary code changes — a policy is closer to a compliance artifact than a feature.

Design choices & alternatives

- **Chosen:** OPA with Rego policies, evaluated via HTTP call from the control plane, **fail closed** on any error (unreachable OPA = denied, not allowed).
- **Rejected:** hardcoded role checks in application logic — faster to write initially, but policy changes require code review + deploy of the whole service, and there's no single place to see "what can an analyst actually do" without reading scattered `if` statements.
- The fail-closed behavior is a specific, deliberate choice: a broken or unreachable policy engine should degrade to "nobody gets access" rather than "everybody gets access."

Impact on the solution

- This is what makes RBAC/ABAC changes ship independently of the SQL execution path (see Section 1) — a new business rule is a Rego change and a policy reload, not a code deploy.
- Fail-closed means an OPA outage causes a service degradation (requests denied) rather than a security incident (requests silently allowed) — a deliberate tradeoff of availability for safety.

Interview questions & answers

Q: Why fail closed instead of failing open when the policy engine is unreachable? A: Because the cost of the two failure modes is wildly asymmetric. Failing open during an OPA outage means every request is authorized regardless of policy — a total access-control bypass exactly when you have the least visibility into what's happening. Failing closed means legitimate users are blocked until the outage is resolved — a real cost, but a bounded and detectable one (you'll see a spike in denied requests, not a silent bypass).

Q: Why OPA specifically, versus hardcoded checks? A: Declarative, versioned, independently testable policy. You can write unit tests against Rego policies exactly like code, and you can diff a policy change in a PR the same way you'd diff any other config change — without touching the service that enforces it.

Q: How would you test a Rego policy in CI? A: OPA ships its own test framework (`opa test`) for exactly this — you write input/expected-decision pairs as Rego test cases and run them in CI before the policy is deployed, independent of the application's own test suite.

---

6. Control Plane: Cost-Tiered Model Router

What it is

A router that picks which model to use per task (classification, SQL generation, synthesis, reranking), with an ordered fallback chain per tier so a single provider outage degrades quality rather than availability.

Why it exists

Not every task needs the most expensive model. Classification (only invoked as a fallback anyway) can use a cheap, fast model; final answer synthesis — the user-facing prose — benefits from the highest-quality model available. Treating "which model" as a single global choice either overpays for cheap tasks or underdelivers on tasks that need quality.

Design choices & alternatives

- **Chosen:** per-task tiers, each with an ordered list of model choices; on failure, walk the chain.
- **Rejected:** single model for every task — simpler, but either wastes money on cheap tasks or under-serves quality-sensitive ones, and has no resilience to a single provider's outage.
- The fallback-chain design means a provider incident doesn't take the system down — it takes quality down one notch, which is a much better failure mode for a user-facing system.

Impact on the solution

- Cost: routing by task complexity is called out explicitly as one of the system's primary cost levers, alongside the classifier and caching.
- Resilience: fallback chains mean "the preferred model's API is down" turns into "this response used a slightly weaker model," not "the request failed."

Interview questions & answers

Q: How do you decide which model tier a task belongs to? A: By what's actually at stake if quality drops — synthesis output is what the user reads directly, so it gets the top of the fallback chain; classification is a fallback path already invoked rarely, so speed matters more than using the top-tier model there.

Q: What happens if every model in a tier's fallback chain fails? A: The router re-raises the last exception — by design, it doesn't silently return nothing. The caller (e.g. the orchestrator) is responsible for turning that into a user-facing degraded response rather than a hard failure, where that's the right choice.

Q: Isn't a fallback chain risky if a cheaper fallback model produces a worse answer without anyone noticing? A: This is a real risk, which is why the guardrail pass (Section 12)

and the golden-dataset evaluation pipeline are meant to run regardless of which model actually produced the answer — quality checks shouldn't assume the top-tier model was used.

---

7. Data Plane: AST SQL Validator

What it is

The core safety mechanism for the SQL path. Every LLM-generated SQL string is parsed into an abstract syntax tree (via `sqlglot`) and run through a five-step decision tree before it's allowed anywhere near the warehouse:

1. Did it parse? If not, reject.
2. Is the root node a `SELECT` (no DDL/DML)? If not, reject.
3. Is every referenced table and column on the allowlist? If not, reject.
4. Does it have a `LIMIT` clause? If not, inject a default one.
5. Is the `EXPLAIN`-based cost estimate under threshold? If not, queue it as an async job instead of running it inline.

Why it exists

The original design left "SQL Validator" as an unspecified box. LLM-generated SQL is untrusted input in the same sense any user input is — it needs a security boundary, not a hope that the model behaves. String/regex-based filtering is a common but fragile choice here because it can be bypassed by comments, string-escaping, or SQL constructs a regex author didn't anticipate.

Design choices & alternatives

- **Chosen:** AST parsing (`sqlglot`), because every check operates on parsed structure — a comment or a creatively encoded string doesn't change what the parse tree actually contains.
- **Rejected:** regex/string-match filtering — faster to write, but a well-known category of bypass exists for essentially every naive string filter (nested subqueries, comments, alternate encodings).
- Two of the five checks are corrective rather than purely rejecting: a missing `LIMIT` is silently injected, and an overly expensive query is queued asynchronously rather than executed — both preserve the user getting *an* answer, just not necessarily an instant one.
- Every rejection returns a specific, loggable reason — useful for debugging, and for noticing repeated attempts to probe the boundary (itself a signal worth alerting on).

- This is the actual authorization boundary for query scope, not just a nice-to-have — the allowlist config (`config/sql_allowlist.yaml`) is described in the codebase as something to review "the same way you would review an IAM policy."
- Removing this and relying on database-level protections alone (read-only role, RLS) still leaves you exposed to a query that's technically read-only but catastrophically expensive, or that reads out-of-scope columns the DB role happens to have access to.

Interview questions & answers

Q: Why use an AST parser instead of just checking the SQL string for forbidden keywords? A: Because keyword-based filtering operates on the surface text, and surface text can be manipulated — comments, whitespace tricks, alternate clause ordering, or a keyword appearing inside a string literal can all fool a naive filter without changing what the query actually does. An AST reflects the parsed *meaning* of the query — a `DROP TABLE` is a `Drop` node in the tree regardless of how it's formatted, so there's no surface-level trick that hides it from a structural check.

Q: Why silently inject a `LIMIT` instead of rejecting queries that don't have one? A: Because a missing `LIMIT` isn't a security problem — it's a cost/performance one, and it's usually just an oversight in generation rather than malicious. Rejecting it outright would degrade the user experience for a harmless mistake; injecting a sane default lets the query still run safely.

Q: Why queue expensive queries instead of just rejecting them? A: A legitimate, correctly-scoped query can still be too expensive to run inline (e.g. a full-table scan across a year of data). Rejecting it outright means a valid question goes unanswered; queuing it async means the user still gets a real answer, just not synchronously — this preserves usefulness while protecting the warehouse from being locked by a single runaway query.

Q: What's the security relationship between this validator and row-level security (RLS) in the database? A: Defense in depth. The validator is the primary authorization boundary — it decides what's allowed to even be attempted. RLS enforced at the database layer is the backstop that holds even if the validator (or the application logic around it) has a bug: the database itself refuses to return rows outside the caller's tenant/scope regardless of what SQL reached it.

Q: How would you extend this validator to catch a new kind of risky query pattern you hadn't considered? A: Add a new step to the decision tree operating on the same parsed AST — for example, detecting cross joins without a join condition (a common cause of accidental cartesian product blowups) by walking `exp.Join` nodes and checking for a missing `on` clause. The pattern of "walk the parsed tree, check a structural property, reject or correct" generalizes to essentially any new rule.

---

8. Data Plane: SQL Executor (read-only role, timeout, circuit breaker)

What it is

The component that actually runs validated SQL against the warehouse, using a read-only database role with row-level security, a per-query statement timeout, and a circuit breaker that opens after repeated failures.

Why it exists

Even validated SQL needs a second layer of protection at execution time, because the validator's correctness is not a substitute for the database's own guarantees — bugs happen, and the database itself should be incapable of executing a write or returning cross-tenant data even if application logic upstream fails.

Design choices & alternatives

- **Chosen:** read-only role + RLS at the database layer, on top of (not instead of) the AST validator.
- **Rejected:** a full-access connection with only application-level checks — this makes the app the *only* thing standing between a bug and a write, which is a single point of failure for something security-critical.
- **Chosen:** circuit breaker around execution, so a struggling warehouse (or a cascading sequence of timeouts) fails fast for subsequent requests rather than piling up slow queries on an already-struggling system.

Impact on the solution

- If the DB role were writable, an application bug (not even malicious input — just a bug) could result in data corruption. The read-only role makes that class of failure structurally impossible, not just unlikely.
- The circuit breaker is what turns "the warehouse is degraded" into "SQL answers are temporarily unavailable" instead of every concurrent request queueing up behind a slow or dead connection, worsening the outage.

Interview questions & answers

Q: If the AST validator already rejects non-SELECT statements, why also restrict the DB role to read-only? A: Because the validator is application code, and application code can have bugs. The read-only role is enforced by the database itself — it's not something a bug in the validator or a new code path that skips validation could ever accidentally bypass. It's the "trust nothing, verify everything, and put a hard backstop under it anyway" principle applied concretely.

Q: How does the circuit breaker decide when to open, and what happens to requests while it's open? A: It opens after a threshold number of consecutive failures and fails fast for a cooldown window rather than letting new requests pile up against something that's

already failing — this protects both the client (fast failure instead of a long timeout) and the struggling dependency (no additional load while it's unhealthy). After the cooldown, it half-opens to test whether the dependency has recovered.

Q: What's the tradeoff of a statement timeout that's too short vs too long? A: Too short and you kill legitimate slow-but-valid queries (a large aggregation that just needs more time), producing false failures. Too long and a single runaway query can hold a connection (and warehouse resources) far longer than it should, worsening contention for everyone else. It's tuned against real query latency distributions, not picked arbitrarily.

---

9. Data Plane: Hybrid Retrieval (BM25 + Vector + RRF + Rerank)

What it is

Document retrieval that runs two independent searches — BM25 keyword search and vector semantic search — and combines them with reciprocal rank fusion (RRF), then reranks and compresses the fused results with a cross-encoder before they reach the synthesizer.

Why it exists

BM25 and vector search fail in different, complementary ways. BM25 catches exact terms — SKUs, account IDs, service codes — that a semantic embedding might blur past. Vector search catches semantic intent ("optimize storage cost") that keyword matching alone would miss if the document doesn't use those exact words. Neither one alone is sufficient for a domain with both structured identifiers and natural-language intent in the same queries.

Design choices & alternatives

- **Chosen:** hybrid BM25 + vector, fused via RRF (a rank-based fusion method that doesn't require the two systems' raw scores to be on comparable scales).
- **Rejected:** vector-only search — simpler, but demonstrably misses exact-match terms in this domain.
- **Chosen:** a cross-encoder/rerank pass after fusion, plus contextual compression (trimming to relevant spans rather than dumping full documents into the LLM's context).
- **Rejected:** no reranking pass, or full-document context dumps — cheaper per call but meaningfully worse precision, and more tokens for the model to potentially hallucinate from.

Impact on the solution

- Precision@k improves meaningfully with the rerank pass over raw bi-encoder ranking — the difference between "roughly relevant chunks" and "the chunks that actually answer the question."

- Contextual compression directly reduces token cost per call and reduces the surface area for hallucination, since the model sees less irrelevant text to potentially misuse.

Interview questions & answers

Q: Why fuse with reciprocal rank fusion instead of just averaging or summing the two systems' raw scores? A: BM25 scores and vector cosine-similarity scores live on completely different, incomparable scales — a BM25 score of 8.2 and a cosine similarity of 0.82 don't mean the same thing, so naively combining them is meaningless. RRF sidesteps this by fusing based on *rank position* within each list rather than raw score magnitude, which makes it robust regardless of how the two underlying systems are scored.

Q: What's the point of reranking after you've already fused two search results? A: Fusion gets you a reasonable candidate set quickly and cheaply, but a cross-encoder reranker can score query-document relevance far more precisely than either BM25 or a bi-encoder vector search, because it looks at the query and document together rather than as independently-embedded vectors. It's more expensive per-pair, which is exactly why it only runs on the small fused candidate set, not the whole corpus.

Q: Why compress context instead of just giving the model the full retrieved documents? A: Two reasons: cost (tokens are the dominant per-call expense) and hallucination surface — the more irrelevant text is in context, the more opportunity the model has to synthesize something plausible-sounding but wrong from text that wasn't actually relevant to the question.

---

10. Data Plane: Hybrid Query Orchestrator

What it is

The flow that handles the hardest case: a question that needs both a SQL answer and document context, correlated into one coherent response. It fans out SQL generation and document retrieval in parallel, merges the results, correlates rather than concatenates them, synthesizes with enforced citations, runs guardrails, then writes to cache and the audit log.

Why it exists

The original design described this flow at the level of "Generate SQL + Retrieve Documents → Merge → LLM Synthesizes" — a single box that hid several separate, independently-failable steps. Making each step explicit (fan-out, SQL path, retrieval path, merge, correlate, synthesize, guardrail, cache, audit) means each one can have its own error handling, rather than one opaque failure mode for the whole hybrid path.

Design choices & alternatives

- **Chosen:** SQL generation and document retrieval run concurrently (`asyncio.gather`),

since there's no dependency between them at that stage — this is a pure latency win.

- **Chosen:** if the SQL path is queued async (too expensive to run inline) or rejected (failed validation), the hybrid flow still proceeds on documents alone rather than failing the whole request — a partial answer with a warning beats no answer.
- **Chosen:** a dedicated correlation step (not just concatenation) — combining "what happened" (structured results) and "what to do" (documentation) requires reasoning about the relationship between the two, not just presenting them side by side.

Impact on the solution

- Fan-out concurrency directly reduces user-facing latency for the hardest query type.
- Explicit, separable steps mean a failure in one part of the flow (say, the SQL path being queued) degrades gracefully rather than taking down the whole hybrid answer.
- This is also where an audit-relevant detail lives: "LLM synthesizes" being decomposed into merge → synthesize → guardrail-check means each of those three has its own accountable, loggable outcome.

Interview questions & answers

Q: Why run SQL generation and document retrieval in parallel instead of sequentially? A: There's no data dependency between them — the SQL query doesn't need the retrieved documents to be generated, and vice versa. Running them sequentially would only add latency with no corresponding benefit, so concurrency is a straightforward win here.

Q: What happens to the hybrid answer if the SQL side gets queued for async execution because it's too expensive? A: The document side still returns normally, and the flow proceeds to correlate and synthesize using just the retrieved documents — the user gets a real answer built from the evidence that was available synchronously, rather than the whole request stalling or failing because one of its two paths couldn't complete inline.

Q: Why have a distinct "correlate" step instead of just handing both result sets straight to the synthesizer? A: Correlation is where the actual value of a hybrid answer comes from — relating *why* the SQL numbers look the way they do to *what the documentation recommends doing about it*. Handing both raw result sets to the synthesizer and hoping it does that reasoning implicitly is less reliable and less auditable than making that correlation an explicit, separately-testable step.

---

11. Synthesizer: Citation Enforcement

What it is

The step that turns correlated results into user-facing prose, with a hard structural rule: every factual claim must be tagged with a citation to either a retrieved document chunk or the SQL result, and any citation that doesn't resolve to something actually retrieved or executed causes the answer to be rejected.

Why it exists

Prompting a model to "always cite your sources" is not the same as *enforcing* it — a model can still produce a confident, well-formatted citation that doesn't actually correspond to real evidence. Verifying citations against the actual set of retrieved/executed evidence turns "please be accurate" from a request into a structural guarantee.

Design choices & alternatives

- **Chosen:** structural verification — every `[cite:<id>]` tag in the output is checked against the real set of valid source IDs; any mismatch raises an error rather than silently shipping.
- **Rejected:** trusting the prompt alone — cheaper to implement, but doesn't actually prevent a fabricated citation from reaching the user, since the model's compliance with instructions is probabilistic, not guaranteed.
- Confidence scoring is a heuristic based on how much of the available evidence was actually used, weighted toward answers that draw on both structured and unstructured evidence — not a claim of statistical certainty, but a useful signal for the user and for downstream monitoring.

Impact on the solution

- This is the primary mechanism that prevents hallucinated citations from reaching the user — not the only one (guardrails add a second, independent pass), but the first and most direct one.
- Removing this means the "citations" in an answer are decorative rather than verifiable, which undermines the entire premise of an assistant that's supposed to be trustworthy for financial/operational decisions.

Interview questions & answers

Q: Why not just trust the LLM to only cite real sources if you prompt it carefully enough? A: Prompting reduces the *frequency* of fabricated citations but doesn't eliminate it — it's a probabilistic mitigation, not a guarantee. Verifying citations against a known-valid set after the fact is a deterministic check: either the ID exists in what was actually retrieved/executed, or it doesn't, regardless of how well-crafted the prompt was.

Q: What do you do when an unsupported claim is detected — just fail the request? A: In the reference implementation, it raises an error that the caller must handle — which could mean falling back to a stripped-down answer using only the verified claims, retrying synthesis, or (worst case) surfacing a "couldn't produce a fully-cited answer" response. The important design point is that it *never* silently ships an answer with a fabricated citation.

Q: How is the confidence score computed, and what does it actually mean? A: It's a heuristic combining whether both structured (SQL) and unstructured (document) evidence were available, and how much of the retrieved evidence was actually cited in the final answer. It's not a calibrated probability of correctness — it's a proxy signal meant to

flag answers built on thin or one-sided evidence, useful for the user and for aggregate quality monitoring, not a formal guarantee.

---

12. Guardrails: Hallucination Re-check + Outbound PII Scan

What it is

A pass that runs *after* synthesis, independent of the synthesizer itself: an independent check that each cited claim actually matches its source text (not just that a citation ID resolves), and a scan of the outbound response for PII before it reaches the user.

Why it exists

Citation enforcement (Section 11) verifies that a citation *ID* is valid — it doesn't verify that the claim attached to that citation is an *accurate paraphrase* of the source. A model can cite correctly but still subtly misstate what the source actually says. A separate, independent verification pass catches that class of error. Similarly, outbound PII scanning exists because a SQL join can surface a column a role shouldn't see even when the semantic layer is supposed to exclude PII-bearing columns — this is a second layer, not a replacement for excluding PII upstream.

Design choices & alternatives

- **Chosen:** guardrails run on the *outbound* text, as a distinct pass from input-side prompt-injection detection — both directions are checked, not just the inbound query.
- **Chosen:** on PII detection, redact rather than block the entire response — preserves usefulness while still protecting the sensitive data.
- The reference implementation's PII patterns are explicitly called out as illustrative (regex) rather than production-grade — a real deployment would use a dedicated PII detection service.

Impact on the solution

- This is the second of two independent hallucination defenses (citation enforcement being the first) — having both means a failure in one doesn't leave hallucinated content with no backstop.
- Outbound PII scanning is a deliberate defense-in-depth layer under an already-supposed-to-be-clean semantic layer — the assumption "PII was already excluded upstream" is treated as fallible, not guaranteed.

Interview questions & answers

Q: If the semantic layer is supposed to exclude PII-bearing columns already, why scan for PII again downstream? A: Because "supposed to" is an assumption, and assumptions about upstream correctness are exactly what defense-in-depth exists to catch when they're

wrong — a misconfigured join, a new column added without updating the semantic layer's exclusions, or a bug anywhere in that chain. The outbound scan is a backstop that doesn't depend on every upstream step having been done correctly.

Q: Why redact PII rather than reject the whole answer when it's detected? A: Rejecting the whole answer over one PII-bearing field discards otherwise-useful information the user needed. Redaction preserves the useful parts of the answer while still protecting the sensitive field — a better tradeoff for something that's meant to be an assistant, not a strict compliance gate that fails closed on any imperfection.

Q: What's the difference between the hallucination check here and the citation enforcement in the synthesizer? A: Citation enforcement checks that a cited ID is *real* — it exists among what was retrieved/executed. The guardrail's hallucination check goes a level deeper: it verifies that the *claim itself*, not just the citation, actually follows from the cited source's content. A model could cite a real source while still paraphrasing it inaccurately; this second, independent pass is what catches that.

---

13. Result Cache

What it is

A cache (Redis) keyed on the normalized query text combined with the requester's data scope (tenant + business unit), so a cached answer can be safely reused across different users within the same scope, but never leaks across scope boundaries.

Why it exists

Many queries repeat, especially common questions like "what did we spend last month." Recomputing the full hybrid pipeline (SQL execution + retrieval + synthesis + guardrails) for an identical question from a different user in the same business unit is pure waste. But caching by query text alone would risk serving a cached answer across tenants or business units that shouldn't see each other's data — the scope has to be part of the key.

Design choices & alternatives

- **Chosen:** cache key = hash(scope + normalized query text). Scope is part of the key, not an afterthought filter applied after retrieval.
- **Rejected:** caching by raw query text alone — cheaper to key, but unsafe across tenant boundaries.
- Normalization (whitespace collapsing, lowercasing) increases cache hit rate for trivially-different phrasings of the same question, at the cost of occasionally conflating questions that differ only in a way normalization erases — a tradeoff to monitor, not a free win.

Impact on the solution

- This is called out as one of the system's cost levers directly alongside model routing and the classifier.
- Getting the cache key wrong (e.g. omitting scope) would be a data leakage bug, not just a correctness bug — this is worth treating with the same care as an authorization check, because functionally, it is one.

Interview questions & answers

Q: Why include the requester's scope in the cache key instead of just the query text? A: Because the same question asked by two users in different tenants or business units may have completely different correct answers, and — more importantly — a cached answer built from one tenant's data must never be served to another. Including scope in the key makes cross-scope leakage structurally impossible rather than something the application has to remember to check on every read.

Q: What's the risk of normalizing query text before hashing it for the cache key? A: Over-normalization can cause a false cache hit — two questions that are lexically similar after normalization but semantically different get treated as the same cached answer. This is why normalization here is deliberately light (whitespace and case only, not stemming or semantic canonicalization) — aggressive normalization would increase hit rate but at a real risk to correctness.

Q: How would you invalidate the cache when underlying data changes (e.g. new billing data ingested)? A: A short TTL (the reference implementation uses 15 minutes) is the simplest approach and is often sufficient for a system where "close to real-time" is acceptable. A more precise approach would tag cache entries with the data version they were computed against (the same versioning used in ingestion) and invalidate entries tied to a version that's been superseded — trading implementation complexity for tighter correctness.

---

14. Immutable Audit Log

What it is

An append-only record of every request: what was asked, what SQL ran, what was cited, which model versions were used, whether guardrails passed, and the confidence score — written before the response reaches the user.

Why it exists

Beyond general good practice, this closes a specific requirement the original design had no mechanism for: answering "what did the assistant know when it gave this recommendation on this date." That's not just a debugging convenience — it's frequently a compliance

requirement (SOC2, enterprise audit) and a prerequisite for trusting the system's outputs in any regulated or high-stakes context.

Design choices & alternatives

- **Chosen:** append-only, immutable by design — no `UPDATE`/`DELETE` grants on the underlying store, so a compromised or buggy service can't rewrite history.
- **Chosen:** written before the response reaches the user, so there's no gap where an answer was delivered but never logged (e.g. due to a crash after response, before audit write).
- The reference implementation ships a simple local JSONL sink for dev/test, explicitly described as not production-grade — the point being that the `AuditSink` interface, not the specific backend, is the durable contract.

Impact on the solution

- Without this, "what did the system know when it recommended X" is unanswerable after the fact — a serious gap for anything used in a financial or compliance-sensitive context.
- Immutability is what makes the log trustworthy as evidence, not just as a debugging log — a log that could be edited after the fact doesn't serve an audit purpose.

Interview questions & answers

Q: Why does the audit write happen before the response is returned to the user, rather than asynchronously after? A: To avoid a gap where a response was delivered to the user but the corresponding audit record was lost (e.g. the process crashed right after sending the response but before writing the log). Writing first — or at minimum, ensuring the write is durably queued before the response is returned — closes that gap, at the cost of a small amount of added latency.

Q: What makes an audit log actually "immutable" in practice, versus just "a log table you don't usually update"? A: The store itself should not grant `UPDATE`/`DELETE` privileges to the service account writing to it — immutability needs to be enforced by the storage layer's permissions, not by application convention, because application convention is exactly what a bug or a compromised credential can violate.

Q: What would you log if you wanted to reconstruct exactly why the system gave a specific answer, months later? A: Beyond the query and answer text: the exact model versions used (since model behavior varies by version), every source ID actually cited, the SQL actually executed (not just generated — post-validation, post-LIMIT-injection), the guardrail pass/fail result, and the schema/document version the retrieval and SQL generation were operating against — that last one is what ties an answer back to "what the system knew" at that point in time.

---

15. Async Ingestion Pipeline

What it is

A separate, event-driven pipeline (Kafka/event bus → chunk + embed → object storage / vector index / catalog) that processes new billing data and documents entirely outside the live query request path, tagging every chunk with a schema/document version at ingestion time.

Why it exists

New data and documents need to be processed and indexed, but that processing has no reason to block or slow down a live user query — the two have completely different latency requirements (ingestion can tolerate seconds-to-minutes of delay; live queries cannot). Coupling them means a burst of new documents could degrade live query latency for no good reason. Version-tagging at ingestion time is what makes the audit log's "what did the system know on this date" question answerable at all — without it, there's no way to reconstruct which version of a document or schema was in effect when an answer was generated.

Design choices & alternatives

- **Chosen:** Kafka (or a managed equivalent) to decouple arrival from processing — producers (billing systems, document sources) don't need to wait on or know about the consumer's processing speed.
- **Chosen:** every chunk written to object storage as source-of-truth *before* chunking/embedding is attempted, so raw content is never lost even if the embedding step fails downstream.
- **Rejected:** synchronous ingestion inline with query serving — this was explicitly called out as entirely absent from the original design, and reintroducing it would couple two systems with very different latency/throughput needs.

Impact on the solution

- This is what makes the system's data current without ever blocking live traffic — ingestion volume and query volume scale independently.
- Version tagging is a direct dependency of the audit log's most valuable capability (Section 14) — without it, the audit log can tell you what was asked and answered, but not what the system's knowledge actually was at that moment.

Interview questions & answers

Q: Why use an event bus (Kafka) instead of just writing new documents directly into the vector store as they arrive? A: Decoupling arrival from processing means the system producing new data (billing events, document updates) doesn't need to know or care how fast the downstream chunking/embedding pipeline is — it just publishes an event and

moves on. Without that decoupling, a slow embedding step or a temporary outage in the vector store would create backpressure all the way to the systems generating the data in the first place.

Q: Why write raw content to object storage before attempting to chunk and embed it? A: Because chunking/embedding is a processing step that can fail (bad input, embedding service outage, bug in the chunker) — and if the raw content were never durably stored until *after* that succeeded, a failure there would mean the source data is simply lost. Storing the source-of-truth copy first means processing can be retried or fixed later without needing to re-acquire the original data.

Q: How does version tagging at ingestion actually get used later? A: It's attached to every chunk and referenced in the audit log, so a later question like "why did the assistant recommend X on this date" can be answered precisely: by looking up which schema/document version was active at ingestion for the chunks that were retrieved and cited in that specific answer, rather than assuming the current state of the documentation was what the system saw at the time.

---

16. Cross-cutting: Tool selection tradeoffs (quick reference)

These are the tool-choice questions that come up across almost every component above — a condensed reference for interview prep.

Decision	Chosen	Core tradeoff being made
Agent orchestration	Explicit state machine (LangGraph-style)	Supports cycles/retries/branching vs. plain sequential chains, at the cost of more upfront structure
Query routing	Deterministic classifier + LLM fallback	Cost/latency/determinism for the common case vs. flexibility for the rare ambiguous case
SQL validation	AST parsing (sqlglot) + allowlist	Structural correctness vs. the speed of writing a regex filter
DB access	Read-only role + RLS	Defense-in-depth at the DB layer vs. trusting application-only checks
Cost control	EXPLAIN-based estimate + forced LIMIT + timeout	Preventing catastrophic queries proactively vs. discovering them when the warehouse is already locked
Retrieval strategy	Hybrid BM25 + vector	Covering both exact-match and semantic-intent queries vs. the simplicity of one method
Precision boost	Cross-encoder rerank	Meaningfully better precision@k vs. the extra latency/cost of a second-pass model
Context handling	Contextual compression	Lower token cost and less hallucination surface vs. simplicity of full-document context
Access control	OPA (declarative policy)	Reviewable, versioned, independently testable policy vs. hardcoded checks that are faster to write once
Multi-tenancy	RLS + vector namespace + policy (three layers)	Defense-in-depth (no single layer's bug leaks data) vs. the complexity of maintaining three enforcement points
Model resilience	Router + fallback chain	Cost-tiering and outage resilience vs. the simplicity of one model everywhere
Failure handling	Retries + circuit breakers + async queue	Graceful degradation vs. the simplicity of synchronous-only, no-breaker code
Quality gate	Golden dataset + offline eval in CI/CD	Catching regressions before users see them vs. relying on manual QA alone

Interview questions & answers (cross-cutting)

Q: If you had to cut this system's scope in half for an MVP, what would you keep and what would you cut? A: Keep: the control/data plane split, the AST SQL validator, read-only DB access with RLS, and basic authorization — these are the pieces where cutting corners creates security or data-integrity risk, not just a rougher user experience. Cut or simplify first: the deterministic classifier (fall back to LLM-only routing temporarily, accepting the cost), hybrid retrieval (start vector-only), the model router's multi-tier fallback chains (start with one model per task), and the golden-dataset eval pipeline (start with manual QA). The pattern: keep anything that's a safety boundary, defer anything that's primarily a cost or quality optimization.

Q: Where in this design would a bug be most dangerous, and how does the design account for that? A: The SQL path, specifically anything between "LLM-generated SQL" and "the warehouse actually executing it" — this is untrusted-input-to-execution, the classic injection risk surface. The design accounts for it with layered defense: AST validation (structural correctness and scope), a read-only DB role (can't write even if validation is bypassed), row-level security (can't cross tenant boundaries even with a valid read), and a cost estimator (can't lock the warehouse even with a valid, in-scope, read-only query). No single layer is trusted alone.

Q: How would you extend this architecture to support a new data source (e.g. a second warehouse, or a new document type)? A: For a new warehouse: extend the SQL allowlist config and the executor's connection factory, without touching the validator's decision logic itself — the AST-based checks generalize across SQL dialects (sqlglot supports many). For a new document type: extend the ingestion pipeline's chunker/embedder as needed for that format, tag it with its own schema/document version the same way existing types are, and it flows through the same retrieval/rerank path without changes there — the interfaces (`Protocol` classes throughout the codebase) are what make this kind of extension additive rather than invasive.